ACADEMIC CENTRAL

PRACTICE EXAMINATION

NC(V) LEVEL 4

TECHNICAL PROGRAMMING

PRACTICE EXAM 3 · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central NC(V) Level 4 Technical Programming practice exam 3.

Written by Academic Central for practice, following the NC(V) subject guidelines for NC(V) Level 4. It is not a Department of Higher Education and Training marking guideline.

How to mark: each ✓ is one mark. Accept any correct code that does the same job, including other variable names and other correct Python methods. Do not penalise small spelling slips in output text. Follow through (consistent accuracy) where an earlier error is carried forward.

SECTION A

QUESTION 1

1.1 1.1.1 A ✓ 1.1.2 C ✓ 1.1.3 B ✓ 1.1.4 D ✓ 1.1.5 A ✓ 1.1.6 B ✓ 1.1.7 D ✓ 1.1.8 C ✓ 1.1.9 A ✓ 1.1.10 B ✓ (10)

1.2 1.2.1 True ✓ 1.2.2 False ✓ 1.2.3 True ✓ 1.2.4 False ✓ 1.2.5 True ✓ 1.2.6 False ✓ 1.2.7 True ✓ 1.2.8 True ✓ 1.2.9 False ✓ 1.2.10 True ✓ (10)

1.3 1.3.1 C ✓ 1.3.2 E ✓ 1.3.3 B ✓ 1.3.4 H ✓ 1.3.5 A ✓ 1.3.6 D ✓ 1.3.7 F ✓ 1.3.8 G ✓ 1.3.9 J ✓ 1.3.10 I ✓ (10)

TOTAL SECTION A: [30]

SECTION B

QUESTION 2

2.1 (a) int ✓ (b) str ✓ (c) float ✓ (d) bool ✓ (4)

2.2 2.2.1 arm-servo-07 ✓ 2.2.2 ARM-SERVO-07 ✓ 2.2.3 ['arm', 'servo', '07'] ✓ 2.2.4 arm ✓ 2.2.5 1 ✓ (only the "o" in servo; "07" contains the digit zero, not the letter o) (5)

2.3 input() returns a string, and a string cannot be added to an integer (TypeError) ✓✓. Correct line: new_speed = int(speed) + 10 ✓ (float(speed) also accepted) (3)

2.4 Correct f-string ✓, correct output with both variables ✓: (2)

```
print(f"Motor {motor} runs at {rpm} rpm")
```

[14]

QUESTION 3

3.1 A list can be changed (items added, removed, changed); a tuple cannot be changed once created (immutable) ✓✓. Example, any ONE ✓: fixed coordinates of a home position (x, y, z); RGB colour values; days of the week. (3)

3.2 3.2.1 [12, 15, 9, 15, 20, 7] ✓ 3.2.2 20 ✓ 3.2.3 20 ✓ 3.2.4 [7, 9, 12, 15, 20] ✓ 3.2.5 [12, 9, 15, 20, 7] ✓ (only the first 15 is removed) 3.2.6 5 ✓ (6)

3.3.1 battery["R2"] = 60 ✓ (1)

3.3.2 battery["R4"] = 100 ✓ (1)

3.3.3 Loop over the dictionary ✓; access the value ✓; correct condition < 20 ✓; print the name ✓: (4)

```
for name, level in battery.items():
    if level < 20:
        print(name)
```

[15]

QUESTION 4

4.1 def with the correct name ✓; default diameter=0.1 ✓; import math / use math.pi ✓; correct formula ✓; return ✓: (5)

```
import math

def rpm_to_speed(rpm, diameter=0.1):
    speed = math.pi * diameter * rpm / 60
    return speed
```

4.2 rpm_to_speed(300) ✓ rpm_to_speed(300, 0.15) ✓ (print(…) or assigning to a variable accepted) (2)

4.3 Input ✓; open in append mode "a" ✓; write the line with a newline ✓; file closed / with statement ✓: (4)

```
seconds = input("Enter delay in seconds: ")
with open("settings.txt", "a") as f:
    f.write("Delay set to " + seconds + " s\n")
```

4.4 try block around the risky code ✓✓; except ValueError with a message ✓✓; except ZeroDivisionError with a message ✓✓: (6)

```
try:
    pulses = int(input("Pulses counted: "))
    per_rev = int(input("Pulses per revolution: "))
    print("Revolutions:", pulses / per_rev)
except ValueError:
    print("Please enter whole numbers only.")
except ZeroDivisionError:
    print("Pulses per revolution cannot be 0.")
```

4.5 A syntax error breaks the rules of the language (e.g. a missing colon or bracket), so the program does not run at all ✓. A runtime error happens while the program is running (e.g. dividing by zero, opening a missing file) ✓. (2)

[19]

QUESTION 5

5.1 A class is a blueprint / template that describes the attributes and methods ✓, e.g. a Robot class with name, battery and move() ✓. An object is one actual instance created from the class ✓, e.g. r1 = Robot("Rover") ✓. (4)

5.2 5.2.1: def __init__(self, name, battery=100) ✓, self.name ✓, self.battery ✓. 5.2.2: method with self and distance ✓, battery reduced ✓, print name and level ✓. 5.2.3: correct condition ✓, returns True / False ✓. (8)

```
class Robot:
    def __init__(self, name, battery=100):
        self.name = name
        self.battery = battery

    def move(self, distance):
        self.battery = self.battery - distance
        print(self.name, "battery:", self.battery)

    def needs_charge(self):
        return self.battery < 20
```

5.3 class Drone(Robot) ✓; __init__ with the parameters ✓; super().__init__(name, battery) ✓; self.altitude = 0 ✓; climb method ✓ that increases the altitude ✓: (6)

```
class Drone(Robot):
    def __init__(self, name, battery=100):
        super().__init__(name, battery)
        self.altitude = 0

    def climb(self, metres):
        self.altitude = self.altitude + metres
```

5.4 Any TWO, ✓✓ each: code is reused, so it does not have to be written again; less code means fewer mistakes and easier maintenance (a fix in the parent class fixes all children); new types of robots can be added easily by extending an existing class; classes are organised in a logical hierarchy. (4)

[22]

TOTAL SECTION B: [70]

TOTAL: 100

END OF MARKING GUIDELINE